

Leveraging deep reinforcement learning for dynamic NPC behavior and enhanced player experience in unity3d

Ahmad Affandi Supli^{*} , Xu Siqu

Digital Media Technology Department, School of Computing and Data Science, Xiamen University Malaysia, Jalan Sunsuria, Bandar Sunsuria, 43900 Sepang, Selangor, Malaysia

ARTICLE INFO

Keywords:

Game AI
Machine learning
Unity game engine
Deep reinforcement learning
Player experience

ABSTRACT

With the innovation of information technology and the improvement of computer hardware and software, players' needs gradually change from pursuing the ultimate visual and auditory feast to the inner performance, gameplay, interactive elements, etc. As an essential part of game content, game AI plays the role of communication and interaction with players. Since the emergence of game AI, it has been paid attention to by game developers. However, it is also a difficult job to make brilliant game AI. The common approaches to implementing game AI are finite state machines and behavior trees. Still, these two approaches require much work to implement flexible game AI and are difficult to maintain later. Therefore, this paper aims to investigate machine learning to train a compliant and flexible game AI. Specifically, this paper adopts a method to train game AI in Unity scenes using machine learning methods such as deep reinforcement learning with the help of the ML-Agents toolkit and Python programming interface. In order to better test the performance of the machine learning method, this study designs a game based on the Unity game engine, which includes a game AI implemented using behavior trees and machine learning. Through the production process and the final implementation results, this paper compares the differences in design ideas and implementation process between using behavior trees and using machine learning to implement game AI, as well as the advantages and disadvantages of each. The purpose of this study is to promote machine learning in the field of game research and achieve higher operational efficiency. Relevant existing principles and guidelines inform the game design process. In addition, the game proposed in this paper can be used as future research to promote machine learning in games to achieve a more efficient, more straightforward design and better player experience. After the game implementation, a quantitative research method is used to measure the players' immersion in the game and the players' satisfaction with the designed game AI. The evaluation results showed that most respondents believed that the proposed agents performed well against both human players and inbuilt game agents.

1. Introduction and research motivation

Due to advancements in artificial intelligence, the hardware and software industries have seen a revolution and abrupt transformation in the past several decades. We see this transformation everywhere: in the use of household robot servants, in healthcare, in electronic transactions, in remote sensing, in expert systems, in traffic control, in self-driving cars, and in everything from behavioural algorithms to suggestive search. The development of graphics processing units [1], tensor processing units [2], and deep neural networks [3] has made it possible to implement artificial intelligence in video games.

According to a report from NewZoo, the games market will grow + 8.7 % to reach \$218.7 billion in 2024 [4]. This shows that the game industry is on the rise. The intelligent development of character control systems in game development has gradually become the focus. Video games require the processing and execution of vast quantities of visual data in the form of frames, or more particularly, vast quantities of multidimensional data [5]. However, the training method adopted by game artificial intelligence still has some drawbacks. How to improve the method to achieve more intelligent game artificial intelligence training results is an important research direction. In addition, video games are ideal testbeds for artificial intelligence methods and are

^{*} Corresponding author at: Digital Media Technology Department, Xiamen University Malaysia, Jalan Sunsuria, Bandar Sunsuria, 43900 Sepang, Selangor, Malaysia.

E-mail addresses: ahmadaffandi@xmu.edu.my (A.A. Supli), dmt1909206@xmu.edu.my (X. Siqu).

<https://doi.org/10.1016/j.entcom.2025.101007>

Received 29 July 2023; Received in revised form 17 July 2025; Accepted 22 July 2025

Available online 23 July 2025

1875-9521/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

becoming an increasingly important application area [6].

The need for computers to handle complex algorithms such as pathfinding and decision making with limited computer computing power has led to the emergence of a new approach to game AI design, namely finite state machines. However, this approach leads to the drawback of lack of reusability and flexibility of finite state machines in game development. To reduce the impact of the drawbacks of finite state machines, component-based hierarchical state machines were proposed [7]. This approach alleviates the complexity of the FSM design game AI, but does not solve the problem that finite state machines cannot be in more than one state.

In contrast to the above approaches, the behavior tree approach was introduced and was specifically applied in the game Halo2 [8]. This approach allows a clear logic between individual states, improving reusability and scalability. However, the performance of game AI has always been limited due to the lack of learning mechanism in the behavior tree. The practice of applying machine learning techniques to game development also continues to advance the development of game artificial intelligence.

Training the intelligences in Atari games with machine learning techniques and procedures resulted in significant improvements across 49 different Atari games. However, the intelligences [9] in most of these Atari games are entirely transparent because the games are played in 2D. The application of such machine learning techniques and approaches to 3D settings is currently crucial. Unity's ML Agents toolkit provides an example project called 'Soccer Twos', which applied Reinforcement learning with Proximal Policy Optimization algorithm.

1.1. The academic motivation of AI in games

There is a continuous need to study and employ machine learning (AI) techniques and algorithms to research games due to the enormous potential and role of machine learning in game creation and development. In order to achieve human-level performance and behavior in video games, machine learning is used to train in-game agents or bots. As a result of these improvements, human players will have a better experience and a deeper appreciation for the game. When going up against a human player with skill, an AI agent or bot needs to be able to read the player's actions and emotions. Machine learning scientists and game developers use techniques like reinforcement learning and supervised learning (with tools like support vector machines and neural networks) to create and train models that can better compare and compete with human players in a variety of games. The development and expansion of the computer gaming industry rely heavily on these breakthroughs.

In this research, the influence of different training methods on the results is demonstrated based on a review of the existing literature. Various approaches are observed in research on game AI methods. For instance, a study [10] using a finite state machine (FSM) is effective in simpler environments like the StarCraft game for its simplicity and ease of modification; however, there are limitations in adaptability as more states are introduced. On the other hand, behavior tree methods have been used to solve this deficiency by incorporating additional algorithms such as genetic programming, which have been employed in tower defense and Mario AI games to boost performance. These methods, however, require significant computational power and still lack scalability [11,12].

Machine learning techniques, especially reinforcement learning (RL), enhance the adaptability of agents and reduce training time. However, these studies often have limited applicability because the games used during experimentation were not highly complex and did not fully represent real-life scenarios or player interactions [13–15].

In contrast to supervised learning, which requires large labeled datasets and is less effective in dynamic game environments, deep reinforcement learning (DRL) allows agents to learn autonomously by interacting with the environment and maximizing rewards over time.

Unlike evolutionary algorithms, which are computationally expensive and less suited for real-time decision-making in complex games, DRL enables agents to continuously adapt and improve their decision-making abilities, making it particularly suited for games that require dynamic, real-time interactions with human players. This research demonstrates that DRL offers a powerful approach for training intelligent and adaptive game AI, overcoming the limitations of other traditional machine learning techniques.

A game AI design method combining behavior trees and machine learning is proposed in this study. By fusing behavior trees with machine learning, it aims to generate more responsive AI agents capable of navigating more challenging environments. This results in a more engaging player interaction than conventional game AI designs. This study explores the training methods of game artificial intelligence and brings the theory into practice. The evaluation results of this study provide an effective reference for researchers in related fields.

2. Research on current Game-AI

2.1. Finite state machine

A finite state machine is a state model that has a certain number of states inside it. It can operate at a given time based on inputs, causing a state to transition to another state, or inducing an output or a behavior. The basic concepts in a finite state machine are states, events, transitions, and behaviors. Finite state machines divide the behavior of an object into multiple "states" that are easy to understand and implement. The advantage is that the thought pattern is easy to understand and fast to execute.

However, finite-state machines lack parallelism and can only be in one state simultaneously. Players may thus find behavioral weaknesses in NPCs and therefore predict the outcome of NPC decisions, which leads to a lack of flexibility in NPCs. While the hard-coded property brings the advantage of fast execution, it will also lead to a high degree of code coupling, making it challenging to design its logic directly. And as the state increases, the production difficulty will also increase [16].

2.1.1. Existing game for finite state machines method

Finite state machines and behavior trees are common ways to design game AI. The disadvantages of finite state machines are as follows: The first problem is that the behavior of finite state machines is easy to predict. Since state transitions are performed under the satisfaction of conditions, the behavior of the state machine becomes predictable when the design of the conditions in the code is inferred by the user based on the user. Secondly, Finite state machines become harder to understand as the number of states and transitions between them increases [17]. In addition, Finite state machines do not enable the direct design of their logic. This also means that it is not conducive to the addition of new logic and is not friendly to scaling [11].

Ivan & Sergey [10] proposes mathematical model to win the StarCraft game, which is a 2d real-time strategy game, by extending finite state machines with decision trees, which simplify the FSM at the level of describing the conditions of transitions. However, the solution can't bring the fundamental changes. Author programmed and tested the proposed solution using the method of common automated testing scenarios design [18]. The result verifies that finite state machines have the characteristics of being simple to implement and easy to modify in a simple game environment setting through video game StarCraft. However, due to the small number of behaviors, this also leads to the agent's behavior is more homogeneous and easier to be predicted. Adding states leads to an increase in the degree of relationship between states, which also increases the difficulty of maintenance and modification later.

2.2. Behavior tree

A behavior tree is a multinomial tree structure containing logical and

behavioral nodes. It executes each node in a tree traversal manner, using conditional nodes to determine state changes, and finally behavioral nodes to execute the decision results. Developers can extend it depending on the development use. To meet the needs of complex behavior tree design, it can also include some interruption mechanisms [19].

The tree-like organization of the behavior tree makes it logically clear in its design. Execute each node in a tree traversal manner. A state change is determined using conditional nodes. Finally, the behavior node executes the decision result. This design logic fundamentally gets rid of the drawback of finite state machines that need to define the way of transition between each state. By using nodes with low-level functionality to create highly intelligent NPCs, this modular approach not only makes it easier for technicians to add and rewrite functional requirements but also increases reusability. By implementing randomized nodes and weight probability nodes, the purpose of improving the flexibility of behavior trees can also be achieved [20].

2.2.1. Existing game for behavior tree method

Behavior trees provide game designers with rich process control methods and easy modularity, and are a popular behavior decision technique today [21]. However, due to the lack of learning mechanism in behavior trees, there is a bottleneck in the performance of the designed game AI in the game. Also, behavior trees have the following limitations [22]:

- (1) The creation of the behavior tree needs to be done manually.
- (2) The behavior tree design needs to be maintained and updated by the programmer.

In research proposed by Ismael & Pedro [12], for the purpose of solving the problem that behavior tree lacking of learning mechanism, which means that need to add and adjust the node manually, researchers present trained behavior tree, which generated from traces obtained in a game based on programming by demonstration. The testbed is a tower defense game, which consists two stages: tower deployment first and defense afterward. The AI in this game is a big robot named TowoT, which could fire heavily to help player. In addition, every fire consumes energy. TowoT only had four possible behaviors: protect the spaceship, protect the mining tower, protect Jacob, and recharge itself. Automated metrics are used to measure the performance of the system in creating behavior by conducting user surveys. The result shows that this method promote game designers authoring control on game AI. But it still contains the drawbacks in the scalability and the implementation of the training environment.

In 2019, [23] proposed a method combined behavior tree and genetic programming, which could automatically synthesize a successful behavior tree (BT) in an a priori unknown dynamic environment. The testing environment is Mario AI, which is an open-source software clone of Nintendo's Super Mario Bros. By measuring the performance, in terms of the reward function $p(x)$, and the complexity of the learnt solution in terms of the number of nodes in the behavior tree and Finite state machine, respectively, the result indicates that the proposed approach generated solutions with significantly fewer nodes than the FSM approach. But this method requires good computational power and large number of examples to achieve a high performance.

In summary, on the other hand, focuses on behavior trees and tries to cope with the lack of learning mechanism of behavior trees with other algorithms respectively to solve the problem that designers cannot directly modify the logic of behavior trees, but the limitations of their proposed methods still cannot be fundamentally solved.

2.3. Machine learning

Although the behavior tree solves many drawbacks of finite state machines, it still has shortcomings. For example, the developer needs to

manually code each node in the behavior tree, which is a tedious workload. The behavior tree lacks a learning mechanism and cannot adjust its structure according to the changing information of the game environment. All the behaviors of NPCs are fixed, so flexibility is still a bottleneck. Thus, this study proposed the method based on machine learning. Machine learning research focuses on how to make computers simulate or implement human learning behaviors to acquire new knowledge or skills and reorganize existing knowledge structures to continuously improve their performance [24]. Supervised learning, unsupervised learning, and reinforcement learning are the three primary categories of machine learning [28]. The first approach is popular for classification and regression [29], but it requires a considerable amount of labelled data. Labels and a priori classification are unnecessary for unsupervised clustering and association [30]. This research mainly takes the third way, which is reinforcement learning.

2.3.1. Existing game for machine learning method

Research named The Implementation of Reinforcement Learning Algorithm for AI bot in Fighting Video Game [26], which proposed 2D Fighting video Game, is a concrete example of machine learning techniques in game artificial intelligence. Training methods of machine learning such as Reinforcement Learning enhance the agent's ability to adapt to the game environment and perform desired behaviors, and reduce training time. And research named Reinforcement Learning for Physics-Based competitive Games [27], proposed a 2D shooting game, which has proved that the detailed reward function could raise the chance of the agent performing desirable behaviors and avoiding undesirable behaviors. However, the above studies are all based on a relatively simple game environment design.

To be more specific, this means that the scene only contains small number of static obstacles. But in the actual implementation, the situation might be more complex. For example, the factor might also include terrain orientation slope and moving obstacle. The simple game environment could not well test the efficiency of the algorithm in complicated game setting. The performance might be different than the experiment result. In addition, the number of actions is relatively small, which not only hard to show the intelligent of the agents but also makes the movement easy to predict. As the consequence of that, it might have poor impact in player engagement experience.

All these issues can have some negative impact on player motivation and engagement. Around these problems and gaps, this study investigates appropriate methods to train game AI and put into practice the related theories by designing a 3D third person shooter (TPS) game.

3. Research on Game-AI based on machine learning method

3.1. Basic objective

The main purpose of this research is to present a third-person shooter with the game AI trained by using the machine learning method. The research objectives are as follows:

- (1) To implement the appropriate elements in AI training model based on the machine learning method in TPS game.
- (2) To evaluate the motivation and player engagement of the proposed prototype.
- (3) To evaluate the performance by doing a comparative analysis between the machine learning method and behavior tree on AI NPC.

To implement the machine learning training method, the research focuses on developing a third-person shooter with multiple different functionality game AI. In the process of designing the game, behavior tree technology, related techniques of machine learning such as reinforcement learning, imitation learning, and methods of combining machine learning results with behavior tree technology are applied and

tested in actual game scenarios. The game is called Mage. The player, as a mage, faces a variety of enemies to attack and resist. Enemies with self-decision systems can be a much richer experience than a normal shooter. This game is a desktop stand-alone offline game, which can be run on windows and mac system.

3.2. Reinforcement learning approaches

Reinforcement learning is one of the important methods in the field of machine learning, which has a good adaptive and learning capability [25]. Its algorithms have been extensively studied and applied in various learning environments [31]. The Reinforcement learning model consists of four parts, which are state, action, reward, and policy.

- (1) State (S) refers to a particular state obtained by an intelligent subject by observing the environment. Through the state value function, the expectation that the states at the moment of time t can be rewarded in the future can be obtained.
- (2) Action (A) reflects the subjective and active influence of the intelligent subject on the environment. The benefit from the action is called the reward.
- (3) Policy (P) is the core of the reinforcement learning idea, which determines the way an agent chooses an action at a specific moment and state and directly affects the execution result of the algorithm. The intelligent subject chooses the appropriate action according to the current state according to a particular policy.
- (4) Reward (R) is the reward and punishment signal obtained by the Agent in the process of observing and influencing the environment. It will be used as a basis for decision-making to influence the next decision [22].

The part of the system other than the intelligent subject feeds back the state, rewards the intelligent agent, and changes according to a particular pattern. The objective of reinforcement learning is to select the state-to-action mapping that yields the greatest benefit.

Creating an ideal policy is reinforcement learning's most worthwhile objective. In this scenario, the function links neural network-based approximation creation to observations. In order to train a policy, this data is subsequently input into a neural network. On-policy and off-policy are the two major definitions of a policy. For on-policy, the same policy is utilised for intelligence-based action selection and training improvement. In order to create and improve the most recent available policy, the agent must interact with the environment [32]. Contrarily, non-policy learning necessitates the creation of a distinct policy for decision-making and improvement [33]. The agent gathers the replay buffer's worth of experiences. Later, the policy will be calculated and updated using this historical data. The cutting-edge Soft Actor-Critic (SAC) is one of the potential non-strategic options for this project [34]. As pointed out by [35], the adversary's behavior is regrettably continually changing, which leads to unfavourable dynamics for the playback buffer. The on-policy strategy was selected for this project because it is more dependable and converges more quickly, despite the fact that off-policy is sample efficient (because no new samples are acquired while the policy is altered).

The OpenAI-proposed Proximal Policy Optimisation algorithm [36] is one of the most extensively utilised policy optimisation algorithms. This algorithm's performance is relatively stable and its maintenance and tuning are relatively simple. However, its efficacy is highly dependent on the selection of hyperparameters and necessitates a substantial quantity of data collection. To address these issues, the OpenAI team initially proposed Trusted Domain Policy Optimisation (TRPO) [37], followed by a streamlined and generalised version, PPO. The PPO algorithm relies on specialised tailoring of the objective function in order to minimise the differences between old and new policies. This algorithm outperforms other gradient-based policy algorithms while preserving equilibrium. Utilising stochastic gradient ascent when updating

the trust domain and eradicating the Kullback-Liebler dispersion penalty yields this advantage. Overall, the algorithm is best adapted for this project because it is a continuous domain problem and comparable projects, such as 3D motion [38], Atari [9], etc., have demonstrated its superior performance.

The PPO algorithm improves the sampling efficiency and robustness of the algorithm by selecting a conservative lower bound for the objective function via a straightforward CLIP mechanism, all while eliminating the need to compute the quadratic gradient in the TRPO algorithm's constraint. Without computing the quadratic gradient in the constraint of TRPO algorithm, PPO algorithm selects a conservative lower bound of the objective function through a simple clip mechanism, which improves the sampling efficiency and robustness of the algorithm, and reduces the complexity of hyperparameter selection.

1) Implementation method based on policy probability ratio clip:

$$L^{CLIP}(\theta) = E_t[\min(r_t(\theta)\widehat{A}_t, \text{clip}(\widehat{r}_t(\theta), 1 - \epsilon, 1 + \epsilon)\widehat{A}_t)]$$

Where θ = policy parameter,

$\widehat{E}_t$ = an empirical expectation over time,

r_t = the ratio of the probability under the new strategy to the probability under the old strategy,

$\widehat{A}_t$ = the estimated advantage of time t ,

ϵ = a super parameter,

In the above formula, $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$, the situation where $r_t(\theta)$ leaves $[1 - \epsilon, 1 + \epsilon]$ is eliminated by clip, which means that if r_t causes the objective function to increase to a certain extent, then r_t changes are ignored. Otherwise, when r_t causes the objective to decrease, it is normally used for optimization.

The following loss function is defined if the critic and performer shares certain parameters.

$$L^{CLIP+VF+S}(\theta) = E_t[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](S_t)]$$

In the above formula, the entropy is computed using the coefficients c_1 and c_2 and the entropy scale factor S . Value function loss, denoted as $L_t^{VF}(\theta)$, is a mathematical expression.

3.3. Platforms and tools

The video game is developed using Unity 3D Engine with C# coding. For the implementation of machine learning NPC, the ML-Agents plugin is used, which could combine the Unity Runtime Environment with Machine Learning [39]. This component enables developers to use reinforcement learning to train behaviors through the PyTorch reinforcement learning library. PyTorch allows the use of CPUs and GPUs to train AI intelligence. After training, a neural network (NN) behavior model will be derived, which can be used to control the intelligence in the Unity engine.

3.3.1. Main components of ML-agents

ML-Agents can combine Unity's runtime environment with machine learning, relying on three main components: Learning Environment, External Communicator, and Python API (Fig. 1). The Learning Environment is the game runtime environment created by Unity, including all environment models, agents, etc. The Agent will get the corresponding observation vector from this environment as the basis for training. External Communicator is the module used to communicate between Unity and external Python API. The Python API is a module that exists completely independent of Unity and contains all the algorithms used for machine learning training. The main algorithms used in this study are those related to reinforcement learning.

Unity scenes provide the learning environment for AI to observe, act and learn. With ML-Agents, developers can turn any game scene into a learning environment. If there are many parameters, it is better to

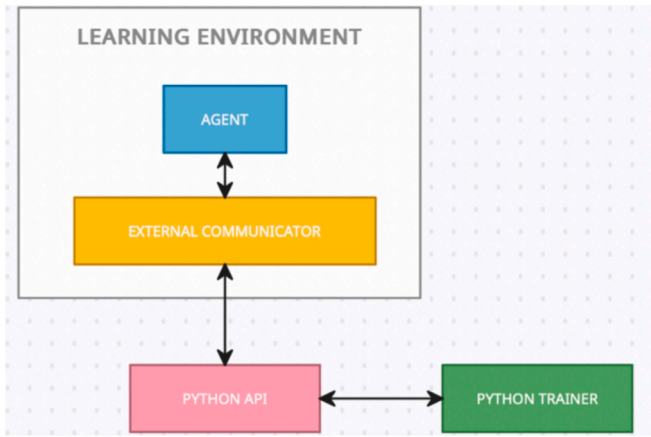


Fig. 1. ML Agent Main Component.

configure the functions in a separate scene for training and then import the model into the original scene for testing after the training is completed.

Python Trainer contains all machine learning algorithms that can train agents, including deep reinforcement learning, imitation learning, course learning, etc. These algorithms are implemented in Python. Python Trainers are only connected to the Python program interface.

The whole operation mechanism is that the agent observes the state from Unity learning. The state is given to an external machine learning algorithm through an external communicator to train the agent. After the policy is generated, the output is passed to the communicator and then to Unity for processing.

3.3.1.1. Training elements of ML-agents. ML-Agents is a reinforcement learning-based plug-in. When using it, three elements necessary in reinforcement learning must be designed, which are state (observations), action, and reward.

Observations: By observations we mean all the necessary vectors that the Agent needs to observe in order to make the best decision. The way of observation can be either numerical or visual form. Digital observations can be classified as continuous or discrete. In most cases, we'll choose to use the continuous approach. In a relatively simple training environment, discrete methods can be adopted. In the ML-Agent plug-in, multiple iteration observation vectors can also be set to achieve the memory effect of the last observation vector. For numeric observation vectors, try to normalize them or limit them to the range -1 to 1. The following formula shows a common normalization procedure.

$$\text{NormalizedValue} = \frac{\text{CurrentValue} - \text{MinValue}}{\text{MaxValue} - \text{MinValue}}$$

NormalizedValue represents the normalized value. CurrentValue refers to the current observation. MinValue is about the minimum value of this observation vector. MaxValue represents the maximum value of this observation vector.

After obtaining the observation vectors, actions are the vectors of instructions output by the policy algorithm. Similar to the observation vector, the action vector can also be continuous or discrete. Usually, when designing an Agent, it is necessary to consider whether the vector controlling the Agent is continuous or discrete.

The reward is an essential part of reinforcement learning. Initially, the Agent does not know what to do but gives some random strategic actions. The reward guides the Agent in which direction it should learn. When the Agent does the right thing, it should be given a positive reward, but this reward should not be excessive; when the Agent does something that it should not do, it should also be given a specific punishment. Appropriate reward settings can be conducive to the Agent's training. To prevent the Agent from finding "loopholes" in the rewards,

it is not advisable to set very complicated rewards.

3.4. Machine learning training details and parameters

In this section, we discuss the machine learning setup used for training the AI agents, particularly with the Proximal Policy Optimization (PPO) algorithm. The training process was carried out using the ML-Agents toolkit in Unity, which integrates machine learning algorithms with the Unity game engine. Below are the key aspects related to training:

(1) Training Parameters.

The key hyperparameters for training were fine-tuned to optimize the performance of the agents. These parameters are summarized in Table 1. The table lists essential parameters like batch size, learning rate, and gamma (discount factor), which were selected based on preliminary experiments and the default configuration from prior use of PPO. For further details on the specific parameters, please refer to Table 1.

(2) Training Duration

The training typically involved 400,000 steps, where each step corresponds to an agent's decision-making process in a simulation. Training sessions generally lasted between 2 and 4 h, depending on the complexity of the training environment and the agent's behaviors. Each training episode involved the agent interacting with the environment to maximize cumulative rewards over time.

(3) Model Validation Methods.

After the training, the following methods were used to validate the models:

Simulation Testing: The trained agents were placed in various game environments to assess their adaptability. Validation involved measuring the agents' survival rates, accuracy in decision-making (e.g., successful attacks or avoidance), and reactions to dynamic obstacles.

Cross-Validation: A form of cross-validation was employed by testing the agents on previously unseen environments to ensure they could generalize well and weren't overfitting to the training data.

User-Centric Evaluation: Finally, player feedback was gathered by having players interact with AI-controlled NPCs in the game. Feedback was collected to assess the realism, challenge, and adaptability of the AI.

These validation methods ensured that the AI models could effectively interact with the dynamic game environment and human players.

3.4.0.1. Vector action

The ML-agents toolkit has an Agent script that developers can extend by creating a new one which inherits from it through it, they can assign rewards and observations to their agents. The Agent script relies on Behavior Parameters script, which communicates observations and rewards to the trainer through the Python API.

- (1) **Vector Observations:** Configure the vector observation space of the agent. By properly configuring the Space Size and Stacked Vectors, we can ensure that the agent receives enough and

Table 1

Hyperparameters for Machine Learning Training Using Proximal Policy Optimization (PPO).

Hyperparameters	Parameter value
batch_size	128
buffer_size	2048
learning_rate	0.0003
beta	0.01
epsilon	0.2
lambd	0.95
num_epoch	3

appropriate information from the environment to better perform its task (shown in Table 2).

- (2) Vector Action: Defining how agents interact with the environment, ML-Agents Toolkit divides Actions into two types: continuous and discrete (shown in Table 3).
- (3) Model: The trained model used by the agent when not training.
- (4) Behavior Type: Define the type of behavior the agent will use. It is divided into three categories: Default, Heuristic, and inference. The default option will train the agent when connected to the Python API, and fall back on inference otherwise. Inference means that the agent is using the trained model to choose its actions. Heuristics are used to test agents.

3.4.0.2. Decision Requester

The DecisionRequester component automatically trigger the agent decision making process and request decisions for an Agent instance at regular intervals. (shown in Fig. 2).

3.4.0.3. Neural network Architecture and characteristics

The neural network used in this study is a fully connected feedforward model designed to process the agent's observations and produce action probabilities. The network consists of an input layer that receives the agent's observations, followed by two hidden layers with 256 neurons each and ReLU activation. The output layer uses softmax activation to generate action probabilities. During training, the network employs backpropagation with cross-entropy loss to adjust the weights based on the rewards received from the environment. The Adam optimizer was used with a learning rate of 0.0003 and a batch size of 128 to optimize performance. After training, the model was evaluated in various environments and demonstrated the ability to adapt to dynamic scenarios, effectively guiding NPC behavior.

3.4.0.4. Training challenges and Mitigation

During the development of the AI agent using deep reinforcement learning, several challenges were encountered. First, the long training time and high computational cost posed difficulties due to the extensive training steps required for convergence. This was addressed by simplifying training environments through the use of polygon-based obstacle layouts and leveraging GPU acceleration via the Unity ML-Agents and PyTorch integration to shorten training duration. Second, reward function design was critical; poorly constructed rewards could lead to ineffective or unintended agent behavior. A balanced reward system was iteratively developed, combining positive reinforcement for successful behaviors and penalties for inefficiencies or undesired outcomes. Third, to ensure the model's generalization to new and dynamic environments, training was conducted across 44 diverse scenarios with varying obstacle placements. Cross-validation and simulation testing further ensured the agent could adapt beyond its original training conditions.

3.5. Environment design

The logical design of the fire demon is to destroy the player by shooting. For each shot at the player, the robot receives a positive bonus. To urge the AI to complete the task faster, a negative bonus is set for the AI that increases with time. To maximize the reward per unit of time, the fire demon will find the player as fast as possible and kill him by shooting him.

Table 2

Vector Observations Description.

Configuration	Definition	Value
Space Size	Define the number of vector dimensions for each observation	4
Stacked Vectors	Define the number of observations stacked per time step.	1

Table 3

Vector Action Description.

Configuration	Definition	Value
Continuous Actions	Define the number of consecutive actions the agent can output at each step.	4
Discrete Branches	Define the number of different discrete action categories that can be chosen at each step	0



Fig. 2. Decision Requester Setting.

Several training scenarios were created to make the fire demon more intelligent and able to adapt to realistic scenarios with multiple obstacles. Each training scenario has a different placement of obstacles. In total, there are 44 training scenes. The obstacles are labelled as “wall.” Only simple polygons were used to simulate the training scenarios. The mechanism is to keep moving toward the agent's position. When the enemy touches the agent, the agent receives a negative reward. To simulate the player's variable displacement, each training scenario has multiple enemies generated in different directions from the agent. To keep generating enemies at the end of each episode, an enemy manager is set up to manage these enemies. After setting up the basic training scene, the next step is the configuration of the agent. The first step is to add two RayPerceptionSensor components to the agent, using rays to simulate the agent's vision (Fig. 3). The rays will record the location information of the enemy and the agent, which is gathered into data to be input into the deep learning algorithm. The next step is to create a script for the agent called ShootingAgent, where the agent's actions, attacks, and rewards (Table 4) are edited. Once the agent is configured, the next step is preparing for training in the python project terminal. Before training, we also have to create a python virtual environment.

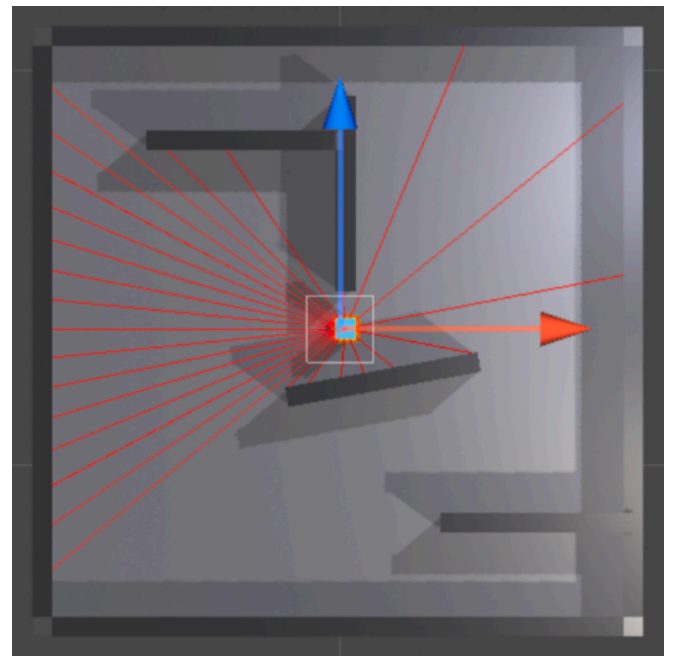


Fig. 3. Example of Training Settings in Unity.

Table 4
Reward Function for the Reinforcement Learning task.

Reward	Value
Enemy Killed	1.0
Damaged	-0.1
Died	-2.0
Collide Wall	-0.01
Distance travelled (1 m/step)	-0.00001

3.6. Instrument design

To ensure an unbiased and more accurate experimental setting, a simple random sampling method is also used in this study. The sampling method is a simple random sampling method, which refers to the equal probability of each situation being included in the sample [40]. Each selection unit is entirely independent of the other without specific correlation and exclusion. In the study of [41], this method is considered one of the fairest and most unbiased sampling methods. And the procedure usually requires less time, which makes information collection more efficient [41]. According to the efficacy analysis of [42], the reliable sample size for game testing should be at least 25–35.

For this experiment, the sample population is mainly college students. The following is the specific experimental method, 40 players were randomly selected and invited to install the game according to their computer systems for personal testing. After playing the whole game, players were assigned to complete a questionnaire.

Descriptive analysis is considered as a constructive identification and summary of data patterns [43]. Descriptive analysis should be used when measuring subjective and sensory data. Therefore, this study also applied this method to analyze players' attitudes toward game participation.

The analysis of the training results is visualized them by generating relevant statistical graphs in the terminal using the Tensor board tool. Different processing is performed for various forms of questions. For demographic questions, since they take the form of single-choice questions, they can be imported directly into the image. This part of the data was analyzed based on frequencies and proportions. During data analysis, the response scores of all questions in the questionnaire based on a five-point Likert scale are combined to generate a composite score that logically measures a single overall characteristic [44]. This is done by calculating the arithmetic mean to obtain the composite score. The formula for its calculation is as follows.

$$\bar{x} = \frac{\sum fx}{n}$$

where: $\bar{x}$ = arithmetic mean,

f = frequency of the answer,

x = value of the answer in Likert Scale.

n = total number of responses.

For the open-ended questions, the meaningless answers such as “no” and “none” are directly eliminated and the remaining answers are presented in the form of a table.

The data is validated during data collection and imported into Microsoft Excel for data analysis to derive player ratings. Throughout the data collection process, a lot of time is spent on data validation. Data validation is very important in ensuring that the data source is true and accurate, which greatly minimizes errors.

The first measure taken in the data validation process was to remove from the test group those respondents who took less than 90 s to complete the questionnaire. This measure aimed to ensure that the players finished the questionnaire after thinking independently and reading the questions carefully. Second, all abnormal responses were removed. The corresponding criteria for anomalies included questionnaires in which all questions in the questionnaire were answered with the same rating. Third, duplicate responses may occur since the questionnaire does not

contain any identifying information to maintain anonymity. To avoid this, the second answer was removed if two identical answers were submitted within 10 min. Finally, after data validation and cleaning, the number of valid questionnaires was 30.

3.7. Specific game and targeted NPCs for AI integration

In this study, the game “Mage” was developed to demonstrate the practical application of AI techniques for NPC behavior. Mage is a third-person shooter game where the player assumes the role of a mage, facing a variety of enemies controlled by AI. The targeted NPCs for AI integration include:

- (1) Fire Demon (Reinforcement Learning-based NPC): The Fire Demon is a central NPC with advanced self-healing and attack skills. It is integrated using reinforcement learning to adapt and respond intelligently to the player's actions in real-time. The Fire Demon uses machine learning to evolve its decision-making processes, adjusting its strategy based on the player's behavior and the in-game environment.
- (2) Sea Demon (Behavior Tree-based NPC): The Sea Demon is designed with a behavior tree to ensure a more predictable yet flexible approach to enemy interactions. It uses a series of pre-defined behaviors to defend its territory, seek shelter when injured, and attack the player when spotted. The Sea Demon represents an NPC with more structured AI logic based on behavior trees.
- (3) Spider Monster (Finite State Machine-based NPC): The Spider Monsters are basic NPCs with finite state machine AI. They exhibit simpler behavior such as attacking the player when within range, and guarding specific areas. The finite state machine controls their behavior, making them less adaptive compared to other NPCs using machine learning-based systems.

These NPCs serve to demonstrate the strengths and weaknesses of different AI approaches, comparing machine learning techniques (like reinforcement learning) with traditional approaches (finite state machines and behavior trees). Each NPC's AI was designed to enhance the player's experience through more dynamic and responsive interactions, showcasing the potential of AI in modern game design.

4. Result analysis

4.1. NPC Description

As shown in the Table 5, there are three different types of game ai in the game.

While the focus of this study is primarily on the implementation and evaluation of different AI approaches (FSM, BT, RL), the selection of 3D models also serves a functional and thematic role within the game's narrative and design. All models used, namely Spider Monster, Sea Demon, and Fire Demon, were sourced from publicly available, royalty-free open-source 3D assets, obtained from the Unity Asset Store. These models were selected based on visual distinctiveness, animation availability, and thematic alignment with the intended behavior of each AI type.

For instance, the Spider Monster (FSM-based) was chosen because its behavior is simple, linear, and aggressive—matching the spider's predatory traits. Its straightforward territorial defense fits well with FSM logic, where transitions are limited and behavior is reactive.

The Sea Demon (BT-based) was chosen to represent a more intelligent and strategic NPC. Sea creatures are often portrayed as elusive and adaptive in mythology and games, which fits well with the behavior tree system where modular and conditional logic defines the demon's tendency to hide and heal. This modular adaptability makes it an appropriate narrative counterpart to BT logic.

Table 5
NPC Description.

AI Design	Method	Characteristic	Mechanism
Spider Monster 	Finite State Machines	Attack: 10 HP Blood volume: 40 HP Number: 6 XP From Kills: 150 Bonus: 200 coins	The mechanism is that when players enter their territory, they will then approach and attack the player. The spider monster's attack method is to bite in close. The only way to get the key to the mysterious door is to kill all of them.
Sea Demon 	Behavior Tree	Regular Attack: 20 HP Reinforced Attack: 40 HP Blood volume: 100 HP Number: 1 XP From Kills: 600 Bonus: 1000 coins	For any intrusion into its territory of outsiders, sea demon will be the first to detect and not hesitate to go to destroy. As the fire demon's assistant, he also inherited the fire demon's ability to repair itself. When his blood level falls below half of his total blood level in the battle, he will seek cover according to his memory of the site and restore his health by temporarily avoiding war. His attack method is by releasing venom in the direction of the aggressor. When every two regular venom balls are released, he can build up his ability to launch a reinforced venom ball.
Fire Demon 	Reinforcement Learning	Regular Attack: 20 HP Reinforced Attack: 40 HP Blood volume: 100 HP Number: 1 XP From Kills: 600 Bonus: 1000 coins	As the leader of the demon race, fire demon has superb self-healing ability and extremely damaging attack skills. He is able to launch two kinds of fireballs. When after every two regular fireballs are launched, he can build up his ability to launch a reinforced fireball.

The Fire Demon (RL-based) is designed to be the final boss, with complex behavior patterns trained via reinforcement learning. Fire symbolizes unpredictability, aggression, and persistence—all features that emerged during training. Using a fire-themed entity to represent a dynamically evolving AI aligns well with the notion of a powerful, learning adversary, reinforcing both gameplay immersion and the conceptual role of reinforcement learning. These visual designs were not arbitrary but intentionally selected to support the game's underlying AI structure and enhance the player's immersive experience. While simpler primitives (e.g., cubes or spheres) could be used for testing RL performance, incorporating recognizable 3D character models elevates the user study's ecological validity, enabling more realistic player feedback on AI immersion, challenge, and satisfaction.

4.2. Machine learning AI

The next section illustrates the appropriate elements of AI training models based on machine learning methods in TPS games by introducing the process of machine learning based AI. The process includes logical design, python project setup, training model design, training result analysis, and model application.

(1) Logic Design.



Fig. 4. Final Result: Model Applied.

The duty of the fire demon is to destroy the player by shooting (Fig. 4). For each shot at the player, the robot receives a positive bonus. To urge the AI to complete the task faster, a negative bonus is set for the AI that increases with time. To maximize the reward per unit of time, the fire demon will find the player as fast as possible and kill him by shooting him.

(2) Training Result Analysis.

As shown in Fig. 5, the training data is represented in a line chart. Y-axis is the cumulative reward for the episode and the X-axis the timestep. The pale lines are raw measurements and the solid lines are smoothed using exponential moving average. Cumulative Reward represents the trend graph of rewards. During the training process, the cumulative reward and time step are recorded for each training set. The expectation is that the average reward increases during the training process and reaches a similar level for each training method. By comparing the average cumulative reward for each episode, it can be seen that the cumulative reward of the AI rises rapidly until 400 k steps of training.

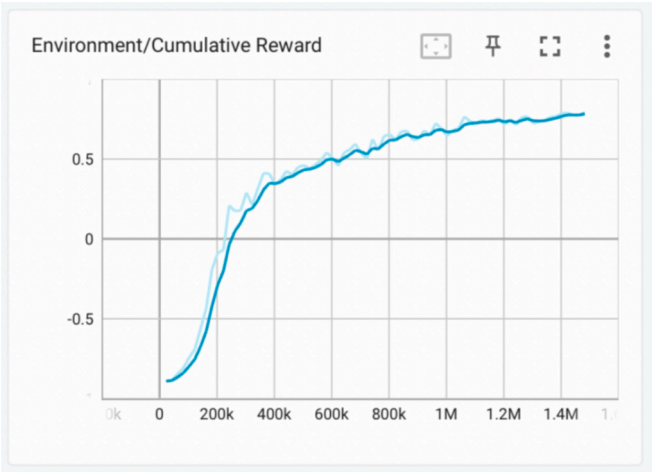


Fig. 5. Training Result: Cumulative Reward.

This is the stage where the AI learns how to obtain more rewards. After 400 k training steps, the rate of accumulated rewards starts to slow down and level off, which indicates that the AI's training has matured and is gradually achieving the desired effect.

Episode Length indicates the training duration of each round and the number of simulation steps required for the agent to complete its goal. The Fig. 6 shows that the episode length decreases as the number of training steps increases, which also means that the agent takes less and less time to complete the task.

4.2.1. Discussion

The questions were divided into six modules according to the game engagement principle: clear game objectives, satisfaction, concentration, perceived availability, challenge, and game AI. The answer pattern of this section was based on the Likert scale to better measure the players' attitudes. Each section contains 4 questions. A score of 1 means "strongly disagree," and five standards mean "strongly agree." The specific scoring criteria are shown in the Table 6.

Overall, looking at each average score, players generally have a positive attitude toward this game prototype. As shown in Fig. 7, the average score for each factor is at least around 4.3. This indicates that this game is well designed and provides a good gaming experience concerning the above game stickiness factors.

From the experimental data, it is clear that most players can easily understand the main content of the game and the player's responsibilities. This is due to the clear instructions for completing the tasks during the game. For satisfaction, the average score of players' evaluation is about 4.2. In testing, players understand the innovative game mechanics and the actual game experience by testing the game themselves, which makes the satisfaction opinion reliable and credible. Regarding whether to keep playing the game, most of players expressed a positive attitude. On the one hand, the AI trained by machine learning can adapt well to the changing environment and react to it in real-time. On the other hand, it can also stimulate the player's sense of excitement, thus making the game more attractive.

The average score for the AI aspect of the game was 4.5. From the data collected, the game AI was the best implemented of all the factors. Part of the reason for this is that the accuracy of the AI attacks is very high, which means that the game is challenging. While the behavioral tree structure AI and the AI trained by machine learning methods scored high overall, the AI trained by machine learning methods was highly recommended by 73.3 % of the test population. This high score also confirms the intelligence of the designed AI. However, an opposing view emerged in investigating whether the game AI was progressively more difficult. After analysis, the reason was derived, the number of spider

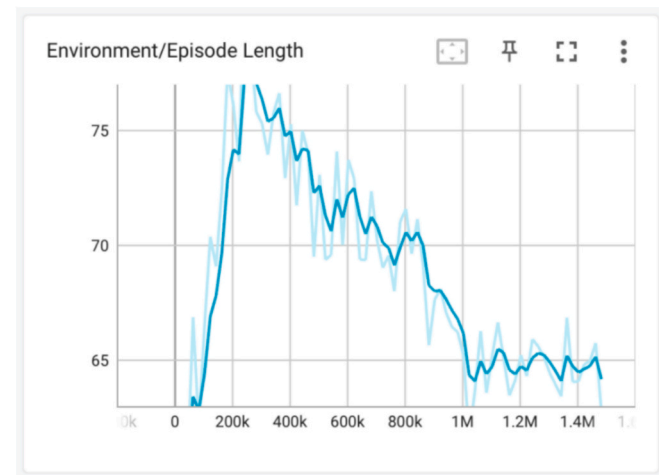


Fig. 6. Training Result: Episode Length.

Table 6

Likert scale Point.

Answer	Strongly Disagree	Disagree	Natural	Agree	Strongly Agree
Point	1	2	3	4	5

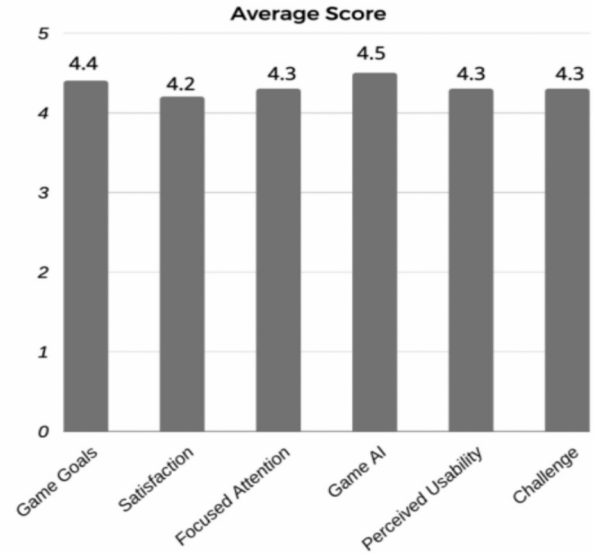


Fig. 7. Average Score.

monsters was too much, and blood relative to the initial skill was high, leading to more difficulty. In the final version of the adjustment, the blood volume of spider monsters was reduced from 100 to 60 to reduce the difficulty of the game AI.

The perceived usability also has a high average score of 4.3. The main factor influencing this high score should be the detailed task reminders and tutorials during the game. The content is straightforward for players to understand. In the survey of whether players felt annoyed during the game and whether the game was demanding for players, about 20 % of the players chose neutral. After analysis, this may be because players need to be more mentally prepared when the enemy starts attacking as soon as they appear. For this, we can add some buffer time for this gap in the later stages. The buffer time can be filled with animation.

To keep players motivated, the challenges in the game should be held moderately tricky. In terms of challenges, mage games achieved an overall average score of 4.3, which shows that the game is relatively difficult to control. Although half of the test players had never played this type of game before, the game's detailed beginner's guide allowed them to get up to speed quickly. And for those unfamiliar with the operation, the game provides prop purchase and random prop generation on the map to balance the game's difficulty.

4.2.2. Managing game difficulty in specialized AI models

To address the potential issue of overly specialized models leading to excessive difficulty, the following methods were used to ensure a balanced experience:

(1) Calibration of NPC Health and Damage:

The Spider Monsters' health was reduced from 100 to 60 to prevent them from becoming too difficult. Similarly, the Fire Demon's self-healing and damage output were adjusted to provide a balanced challenge without overwhelming the player.

(2) Dynamic Difficulty Adjustment:

NPC behaviors were dynamically adjusted based on player

performance to maintain a challenging yet fair experience. As players progress, the AI adapts to ensure continued engagement without becoming frustrating.

(3) Player Feedback:

Continuous playtesting and feedback from players helped fine-tune NPC difficulty. Adjustments were made based on player input to ensure the game remained challenging but not excessively difficult.

4.3. Behavior tree AI

The main task of the sea demon is to destroy the enemies in their domain and the player. The following is an introduction to its specific behavioral functions. The agent is in a dormant state when no enemy is detected. When the agent sees the player, it will attack the player actively. When its blood level is lower than the set threshold, the agent will look for obstacles to hide and wait for its blood level to recover before it attacks the player again.

When there are multiple functional requirements, it is necessary to classify the applicable requirements of each part and determine their priority relationship. For the sea demon, running away after a low blood level belongs to the priority module; the module of attacking enemies belongs to the second module, and chasing enemies is the third priority. The behavior tree node uses the Selector node to handle the priority relationship of each module (see Fig. 8). The priority of the Selector node decreases from left to right. Specifically, the right node has the opportunity to execute only when the higher-priority child node on the left side of the Selector does not satisfy the situation. And all the left child nodes have the function of interrupting the right child node.

The cover processing module is organized internally using Sequence nodes. The sub-node execution method is still executed sequentially from left to right. The subsequent escape node is executed only when the preceding judgment node gets blood below a certain threshold. The specific operation is to stop aiming at the enemy and turn on the accelerated running mode to find the cover. The agent will only end the acceleration once it finds the cover. The agent will hide in the same place until the blood volume has been regained and then continue executing the subsequent nodes. The selector node is used in trying to find the cover node.

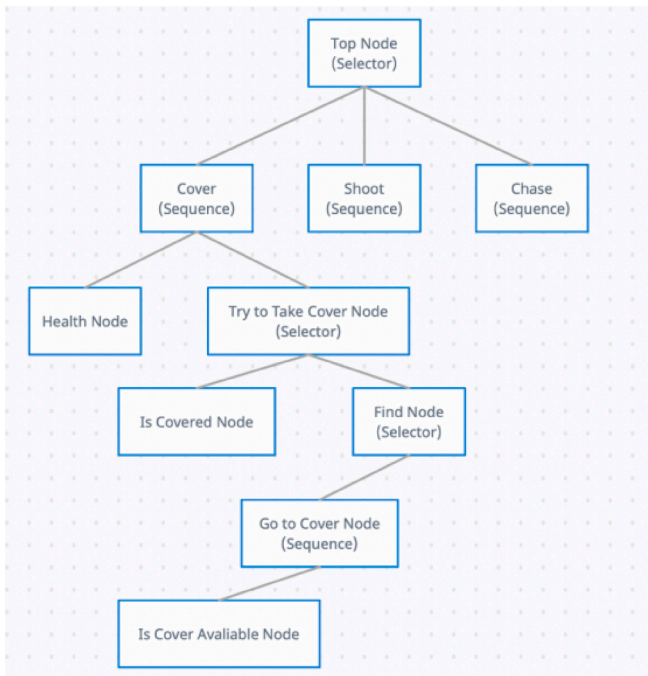


Fig. 8. Behavior Tree Structure.

First, it needs to determine whether the current state is in a covered status, mainly using raycast. If the result is false, run another node and find the node. In the Is Cover Available Node node, in addition to selecting the spot that can achieve the protection purpose from all the covers, it is also necessary to determine the nearest and best hiding point according to the distance. The remaining SHOOT processing module and CHASE module are organized using SEQUENCE nodes. Both of these nodes operate with distance restrictions.

4.4. Comparison of the behavior tree and Machine learning method

In the game development of Mage, both machine learning (ML) and behavior tree (BT) methods were used to implement different NPCs. This section compares the two approaches in terms of design, methodology, and the advantages and disadvantages of each, with a focus on operational efficiency.

The machine learning approach controls NPC behavior through a reward mechanism, allowing the AI to adapt autonomously as it interacts with the game environment. The advantage of this method is that it can be achieved with relatively simple code and leads to more intelligent gameplay behavior. Once trained, ML-based NPCs like the Fire Demon can respond dynamically to new player strategies or environmental challenges, reducing the need for manual updates or constant reprogramming. This provides a significant operational efficiency advantage, especially when compared to traditional methods.

However, one of the disadvantages of machine learning is that it requires considerable training time before results are visible. If the training results are unsatisfactory, adjustments in parameters or algorithms are required, which can prolong the training phase. The design of the algorithm and parameters is therefore crucial when using machine learning methods. A well-planned training approach can save considerable time and effort in the long term, enabling more efficient development.

On the other hand, the behavior tree method controls NPC behavior through a multinomial tree structure with nodes representing actions or conditions. The advantage of this approach is that it provides clear, real-time control of NPC actions based on predefined logic, which is easy to visualize and debug. However, as the complexity of NPC actions increases, the behavior tree design becomes more complex and harder to manage. This can make the maintenance and scalability of behavior trees more labor-intensive.

Another limitation of behavior trees is that they require a navigation bake of the map in advance. This means that NPCs controlled by behavior trees are less adaptable to dynamic game environments, such as moving obstacles or random map generation. In contrast, machine learning-based NPCs can more easily adapt to changing environmental factors. Once the Fire Demon NPC is trained using reinforcement learning (RL), it can be retrained and adapted to new scenarios, improving scalability and allowing for easier integration of new game challenges, such as randomly moving obstacles or procedurally generated maps, which enhance the game's replayability and player engagement.

Unlike traditional methods like finite state machines (FSMs) and behavior trees, machine learning offers significant operational efficiency. With FSMs, NPCs are inflexible, and as the complexity of the game increases, FSMs become more difficult to maintain. Similarly, while BTs offer greater flexibility, they still require manual updates whenever the game environment changes. In contrast, once an NPC is trained with reinforcement learning, minimal intervention is required after the initial training phase. For example, the Fire Demon NPC, trained with Proximal Policy Optimization (PPO), learns to adapt in real-time, allowing for quicker deployment of more complex behaviors without the need for additional code changes. Traditional methods would require significant manual adjustments for each new game scenario, making them more time-consuming and costly to maintain.

In traditional approaches like FSMs or BTs, adding new NPCs, such as

a Spider Monster, or adjusting existing NPC behaviors requires programming new states and reworking behavior logic, which can be labor-intensive. In contrast, once the Fire Demon NPC is trained using RL, it can be easily adapted to new game scenarios by simply retraining the model. This approach reduces the time and effort required to introduce new NPCs or modify existing ones, providing a more scalable solution as the game expands.

5. Discussion and future work

This study lays the groundwork for advanced game AI using machine learning techniques within Unity, particularly in dynamic environments. While the proposed approach has demonstrated its effectiveness in the current setting, there are opportunities for both immediate improvements and broader applications. In the short term, the next steps involve refining NPC behaviors, exploring additional training strategies to enhance their adaptability in more complex environments, and optimizing the training process to reduce computational time without sacrificing AI performance. Additionally, enhancing NPC decision-making by introducing more sophisticated environmental interactions remains a key focus.

Although the implementation in this study utilizes Unity's ML-Agents toolkit, the underlying techniques—particularly reinforcement learning with Proximal Policy Optimization (PPO)—are not inherently tied to Unity and can be adapted for use in other game engines. Game platforms such as Unreal Engine or Godot, for instance, can integrate similar AI training pipelines through external APIs and frameworks like PyTorch or TensorFlow. However, Unity's ML-Agents does streamline many aspects of environment-agent interaction, making it especially convenient for prototyping. Thus, while the Unity-specific toolkit offers certain efficiencies, the methodologies and findings presented in this study can be generalized to other engines with suitable technical adjustments. This expands the potential impact of the research beyond a single platform, supporting broader adoption of intelligent, adaptive NPCs in diverse game development environments.

Looking ahead, several exciting directions for future research emerge. One potential avenue is the direct comparison of machine learning-based AI with human players in terms of decision-making speed, adaptability, and player satisfaction. Although this study evaluates the AI's adaptability and player satisfaction through NPC behavior and user feedback, a direct comparison with human decision-making would provide further insights into how well the AI mimics or surpasses human capabilities in real-time gameplay. Exploring these comparisons could provide valuable benchmarks for improving AI behaviors and designing more intelligent, responsive NPCs that challenge human players in more realistic ways.

In addition, the integration of advanced reinforcement learning algorithms, such as Soft Actor-Critic (SAC), could improve the robustness of NPC decision-making, enabling them to handle more complex, real-time challenges. Meta-learning techniques could help NPCs adapt more quickly to evolving player strategies, providing a more personalized gaming experience. Furthermore, combining machine learning approaches with traditional methods like behavior trees or finite state machines could lead to hybrid AI models that strike a balance between flexibility and predictability.

While the approach is well-suited to dynamic, interactive game genres, it may not be equally effective across all types of games. Games with complex, real-time decision-making, like action or strategy genres, can benefit from this method. However, in simpler games with static or predefined behavior, traditional techniques might suffice. The method's real-time learning capabilities also present a promising direction for games where adaptive AI is essential, allowing NPCs to evolve based on player interactions. Additionally, investigating how emotion recognition and player feedback could dynamically influence NPC behavior to adjust game difficulty in real time holds significant potential for enhancing player immersion. Ultimately, real-time learning, where

NPCs evolve based on ongoing player feedback, could create an ever-evolving, engaging gameplay experience.

Funding

This research was funded by Xiamen University Malaysia Research Fund (XMUMRF) Cycle 9/2022, grant number XMUMRF/2022-C9/IECE/0031.

CRediT authorship contribution statement

Ahmad Affandi Supli: Supervision, Project administration, Investigation, Data curation. **Xu Siqu:** Writing – original draft, Software, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] J.A. Owens, M. Houston, D. Luebke, S.F. Green, J.E. Stone, J.L. Phillips, GPU Computing, *Proc. IEEE* 96 (5) (May 2008) 879–899, <https://doi.org/10.1109/jproc.2008.917757>.
- [2] N.P. Jouppi, et al., In-Datacenter Performance Analysis of a Tensor Processing Unit, *Computer Architecture News* 45 (2) (Jun. 2017) 1–12, <https://doi.org/10.1145/3140659.3080246>.
- [3] M. T. Hagan, H. B. Demuth, and M. D. Beale, *Neural network design*. 1995. [Online]. Available: <http://ci.nii.ac.jp/ncid/BA84139991>.
- [4] NewZoo, *Newzoo Global Games Market Report*, July. 2021, retrieved from Newzoo: <https://newzoo.com/insights/trend-reports/newzoo-global-games-market-report-2021-free-version>.
- [5] K. Adil, F. Jiang, S. Liu, W. Jifara, Z. Tian, and Y. Fu, "State-of-the-Art and Open Challenges in RTS Game-AI and Starcraft," *International Journal of Advanced Computer Science and Applications*, vol. 8, no. 12, Jan. 2017, doi: 10.14569/ijcacs.2017.081203.
- [6] G.N. Yannakakis, J. Togelius, *Artificial Intelligence and Games*. (2018), <https://doi.org/10.1007/978-3-319-63519-4>.
- [7] W. Hu, Q. Zhang, and Y. Mao, *Component-based hierarchical state machine & #x2014; A reusable and flexible game AI technology*. 2011. doi: 10.1109/itaic.2011.6030340.
- [8] D. Isla, *Managing complexity in the halo2 AI system*. In *Proceedings of the Game Developers Conference*, 2005.
- [9] V. Mnih, et al., *Playing atari with deep reinforcement learning*. arXiv preprint arXiv:1312.5602, 2013.
- [10] I.Y. Smolyakov, S. Belyaev, *Design of the Software Architecture for Starcraft Video Game on the Basis of Finite State Machines*. (2019), <https://doi.org/10.1109/eiconrus.2019.8656866>.
- [11] C. Michele, M. Alejandro, "The Advantages of using Behavior Trees in Multi-Robot Systems," *International Symposium* (2016) 1–6.
- [12] I. Sagredo-Olivenza, P.P. Gómez-Martín, M.E. Gómez-Martín, P.A. González-Calero, *Trained Behavior Trees: programming by Demonstration to support AI Game Designers*, *IEEE Trans. Games* 11 (1) (Mar. 2019) 5–14, <https://doi.org/10.1109/tg.2017.2771831>.
- [13] A.R. Abdulla, G. Christos, *Reinforcement Learning for Physics-based Competitive Games*. *International Conference on Innovation and Intelligence for Informatics, Computing and Technologies*, 2020.
- [14] R.A. Adi, A.M. Azliza, H.H. Nurzeatul, O. Rozianawaty, *The Implementation of Reinforcement Learning Algorithm for AI Bot in Fighting Video Game*. 4th International Symposium on Agents Multi-Agent Systems and Robotics, 2021.
- [15] Y.E. Amira, M.E. Sohaila, E.-G.-N. Islam, E.S. Jailan, A.M. Khaled, Y. Khaled, *Building your kingdom Imitation Learning for a Custom Gameplay using Unity ML-agents*. *IEEE 10th Annual Information Technology, Electronics and Mobile Communication Conference. IEEE*, 2019.
- [16] H. Marchand, B. Gaudin, *Supervisory Control Problems of Hierarchical Finite State Machines*. (2003), <https://doi.org/10.1109/cdc.2002.1184677>.
- [17] V.S. Alagar, K. Periyasamy, "Extended Finite State Machine," in *Texts in computer science*, Springer International Publishing (2011), https://doi.org/10.1007/978-0-85729-277-3_7.
- [18] O., C. S., Yu, F. Y., and S., B. A., *Applying of methods and tools to automated testing of telemetric data*, in: *In Izvestiya Etu "LETI"*, 2013, pp. 55–58.

- [19] M. Nicolau, D. Perez-Liebana, M. O'Neill, A. Brabazon, Evolutionary Behavior tree Approaches for Navigating Platform Games, *IEEE Trans. Comput. Intell. AI Games* 9 (3) (Sep. 2017) 227–238, <https://doi.org/10.1109/tciaig.2016.2543661>.
- [20] F. Rovida, B. Grossmann, V. Krüger, Extended Behavior Trees for Quick Definition of Flexible Robotic Tasks. (2017), <https://doi.org/10.1109/iros.2017.8206598>.
- [21] D. Perez, M. Nicolau, M. O'Neill, A. Brabazon, "Evolving Behavior Trees for the Mario AI Competition using Grammatical Evolution." *applications of, Evol. Comput.* (2011).
- [22] J. Lin, J. He, N. Zhang, Application of behavior tree in AI design of MOBA games. *IEEE International Conference on Knowledge Innovation and Invention*, 2019.
- [23] M. Colledanchise, R. Parasuraman, P. Ögren, Learning of Behavior Trees for Autonomous Agents, *IEEE Trans. Games* 11 (2) (Jun. 2019) 183–189, <https://doi.org/10.1109/tg.2018.2816806>.
- [24] T.M. Mitchell, *Mach. Learn.* (1997).
- [25] S.S. Richard, B.G. Andrew, *Reinforcement learning: an Introduction*, MIT press, 2018.
- [26] R.A. Adi, A.M. Azliza, H.H. Nurzeatul, O. Rozianawaty, The Implementation of Reinforcement Learning Algorithm for AI Bot ini Fighting Video Game. *4th International Symposium on Agents Multi-Agent Systems and Robotics*, 2021.
- [27] A. Albuainain, C. Gatzoulis, Reinforcement Learning for Physics-based Competitive Games. *International Conference on Innovation and Intelligence for Informatics, Computing and Technologies*, 2020.
- [28] Taiwo Oladipupo Ayodele, Types of machine learning algorithms, *New Advances in Machine Learning* 3 (2010) 19–48.
- [29] R. Caruana, A. Niculescu-Mizil, An Empirical Comparison of Supervised Learning Algorithms. (2006), <https://doi.org/10.1145/1143844.1143865>.
- [30] Z. Ghahramani, Unsupervised learning, in: *Summer School on Machine Learning*, Springer, 2003, pp. 72–112.
- [31] C. Szepesvári, Algorithms for reinforcement learning, *Synthesis Lectures on Artificial Intelligence and Machine Learning* 4 (1) (2010) 1–103.
- [32] S. Singh, T.S. Jaakkola, M.L. Littman, C. Szepesvári, Convergence results for single-step on-policy reinforcement-learning algorithms, *Mach. Learn.* 38 (3) (Jan. 2000) 287–308, <https://doi.org/10.1023/a:1007678930559>.
- [33] R. Munos, T. Stepleton, A. Harutyunyan, and M. F. Bellemare, *Safe and Efficient Off-Policy Reinforcement Learning*, vol. 29. Cornell University, 2016, pp. 1046–1054. [Online]. Available: <https://arxiv.org/pdf/1606.02647>.
- [34] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, Soft actorcritic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, in: *International Conference on Machine Learning*, 2018, pp. 1861–1870.
- [35] J. Foerster et al. Stabilising experience replay for deep multi-agent reinforcement learning <http://proceedings.mlr.press/v70/foerster17b/foerster17b.pdf> 2017 1146 1155 [Online]. Available.
- [36] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and Oleg Klimov. *Proximal policy optimization algorithms*. arXiv preprint arXiv:1707.06347, 2017.
- [37] J. Schulman, S. Levine, P. Abbeel, M. I. Jordan, and P. Moritz, *Trust Region Policy Optimization*. 2015, pp. 1889–1897. [Online]. Available: <http://proceedings.mlr.press/v37/schulman15.pdf>.
- [38] N. Heess, et al. Emergence of locomotion behaviours in rich environments. arXiv preprint arXiv:1707.02286, 2017.
- [39] A. Juliani, V. Berges, E. Teng, A. Cohen, J. Harper, C. Elion, D. Lange, Unity: A General Platform for Intelligent Agents, arXiv, 2018.
- [40] H. Taherdoost, "Sampling Methods in Research Methodology; How to Choose a Sampling Technique for Research," *SSRN Electronic Journal*, vol. 5, Jan. 2016, [Online]. Available: <https://hal.archives-ouvertes.fr/hal-02546796>.
- [41] G. Sharma, "Pros and cons of different sampling techniques," *International Journal of Applied Research*, vol. 3, no. 7, pp. 749–752, Jan. 2017, [Online]. Available: <https://www.allresearchjournal.com/archives/2017/vol3issue7/PartK/3-7-69-542.pdf>.
- [42] J. M. Davis, K. Steury, and R. J. Pagulayan, "A survey method for assessing perceptions of a game: The consumer playtest in game design.," *Game Studies*, vol. 5, no. 1, Oct. 2005, [Online]. Available: <https://dblp.uni-trier.de/db/journals/gamestudies/gamestudies5.html#DavisSP05>.
- [43] Z.C. Holcomb, *Fundamentals of Descriptive Statistics*, Routledge, 2016.
- [44] A. Joshi, S. Kale, S. Chandel, D. Pal, Likert Scale: Explored and Explained, *British Journal of Applied Science and Technology* 7 (4) (Jan. 2015) 396–403, <https://doi.org/10.9734/bjast/2015/14975>.